

```

using System;

namespace ConsoleApp4
{
    internal class Program
    {

        #region p2

        //public static void TestDefensiveCode()

        //{

        //    int X, Y;

        //    bool flag;

        //    do

        //    {

        //        Console.WriteLine("Write value of X: ");

        //        flag = int.TryParse(Console.ReadLine(), out X);

        //    }

        //    while (!flag || X <= 0);

        //    do

        //    {

        //        Console.WriteLine("Write value of Y (must be > 1): ");

        //        flag = int.TryParse(Console.ReadLine(), out Y);

        //    }

        //    while (!flag || Y <= 1);

        //    Console.WriteLine($"X = {X}, Y = {Y}");

        //}

        ////Prevents Application Crashes Unlike int.Parse() which throws a FormatException or OverflowException on invalid input

        ////int.TryParse() safely handles non-numeric inputs without raising runtime exceptions.

        #endregion

        #region p9

        //public static void SumAndMultiply(int a, int b, out int sum, out int product)

        //{

        //    sum = a + b;

```

```

// product = a * b;

//}

#endregion

#region p10

//public static void PrintMessage(string message, int n = 5)

//{

//  for (int i = 0; i < n; i++)

//  {

//    Console.WriteLine(message);

//  }

//}

////Because the C# compiler resolves arguments by positional alignment

#endregion

#region p13

//public static int SumArray(params int[] numbers)

//{

//  int total = 0;

//  foreach (int num in numbers)

//  {

//    total += num;

//  }

//  return total;

//}

#endregion

static void Main(string[] args)

{

    #region p1

    //try

    //{

```

```

// Console.Write("Enter the first number: ");

// int num1 = int.Parse(Console.ReadLine());


// Console.Write("Enter the second number: ");

// int num2 = int.Parse(Console.ReadLine());


// int result = num1 / num2;

// Console.WriteLine($"Result: {num1} / {num2} = {result}");

//}

//catch (DivideByZeroException)

//{

// Console.WriteLine("Division by zero is not allowed.");

//}

//finally

//{

// Console.WriteLine("Operation complete");

//}


//// Code inside the finally block is guaranteed to execute regardless of whether an exception occurs or is caught in the try-catch block

////It is used to release unmanaged resources such as database connections, files

#endregion


#region p3

//int? number = null;

//int result = number ?? 0;

// if (number.HasValue)

// {

// Console.WriteLine(number.Value);

// }

// else

// {

// Console.WriteLine("number is currently null");

// }

// //Attempting to access the .Value property of a nullable type when .HasValue is false throws an InvalidOperationException

```

// //because the nullable object does not hold a valid underlying value To prevent this exception, always check .HasValue first or use the null-coalescing operator (??)

#endregion

#region p4

//int[] numbers = { 10, 20, 30, 40, 50 };

// try

// {

// int value = numbers[10];

// Console.WriteLine(\$"Value:{value}");

// }

// catch (IndexOutOfRangeException ex)

// {

// Console.WriteLine(ex.Message);

// }

// //Prevents Runtime Crashes Accessing an index outside the valid range throws an IndexOutOfRangeException which can crash the application if unhandled

#endregion

#region p5

//int[,] matrix = new int[3, 3];

//for (int i = 0; i < matrix.GetLength(0); i++)

//{

// for (int j = 0; j < matrix.GetLength(1); j++)

// {

// bool flag;

// do

// {

// Console.Write(\$"Enter element at index [{i},{j}]: ");

// flag = int.TryParse(Console.ReadLine(), out matrix[i, j]);

// } while (!flag);

// }

//}

////GetLength(0) returns the number of Rows(1st dimension).

////GetLength(1) returns the number of Columns(2nd dimension).

#endregion

#region p6

//int[][] jaggedArray = new int[3][];

//jaggedArray[0] = new int[2];

//jaggedArray[1] = new int[4];

//jaggedArray[2] = new int[3];

//for (int i = 0; i < jaggedArray.Length; i++)

//{

// Console.WriteLine(\$"\\nEntering elements for Row {i + 1} (Size: {jaggedArray[i].Length}):");

// for (int j = 0; j < jaggedArray[i].Length; j++)

// {

// bool flag;

// do

// {

// Console.Write(\$"Element [{i}][{j}]: ");

// flag = int.TryParse(Console.ReadLine(), out jaggedArray[i][j]);

// } while (!flag);

// }

//}

//for (int i = 0; i < jaggedArray.Length; i++)

//{

// Console.Write(\$"Row {i + 1}: ");

// for (int j = 0; j < jaggedArray[i].Length; j++)

// {

// Console.Write(\$" {jaggedArray[i][j]}\\t");

// }

// Console.WriteLine();

//}

////Rectangular Arrays:Allocated as a single contiguous block of memory in the Heap

////Jagged Arrays:Allocated as an array of references pointing to separate 1D sub - arrays in the Heap non-contiguous memory

```
#endregion
```

```
#region p7
```

```
//string ? name = null;
```

```
//Console.Write("Enter your name:");
```

```
//name = Console.ReadLine();
```

```
//int length = name!.Length;
```

```
////Purpose of Nullable Reference Types in C#:Prevents NullReferenceException
```

```
#endregion
```

```
#region p8
```

```
//int number = 100 ;
```

```
//object boxedObject = number;
```

```
//int unboxedNumber = (int)boxedObject;
```

```
//try
```

```
//{
```

```
// double invalidCast = (double)boxedObject;
```

```
//}
```

```
//catch (InvalidCastException ex)
```

```
//{
```

```
// Console.WriteLine(ex.Message);
```

```
//}
```

```
////boxing High Memory Overhead ,Unboxing requires type-checking at runtime to verify data safety
```

```
#endregion
```

```
#region c9
```

```
//SumAndMultiply(10,5,out int sumResult, out int productResult);
```

```
//Console.WriteLine($"Sum:{sumResult}");
```

```
//Console.WriteLine($"Product:{productResult}");
```

```
////Because the C# compiler enforces definite assignment.
```

```
////It treats out parameters as unassigned variables and forces the method to assign values to them before returning
```

```
#endregion
```

```
// PrintMessage(n:3, message: "Message");
```

```
#region p11
```

```
//int[] ? numbers = null;
```

```
//int? length = numbers?.Length;
```

```
////it checks if the object on its left side is null before accessing the member on its right
```

```
#endregion
```

```
#region p12
```

```
//Console.Write("Enter a day of the week ");
```

```
//string inputDay = Console.ReadLine();
```

```
//int dayNumber = inputDay switch
```

```
//{
```

```
//  "Monday" => 1,
```

```
//  "Tuesday" => 2,
```

```
//  "Wednesday" => 3,
```

```
//  "Thursday" => 4,
```

```
//  "Friday" => 5,
```

```
//  "Saturday" => 6,
```

```
//  "Sunday" => 7,
```

```
//  _ => -1
```

```
//};
```

```
//Console.WriteLine($"Day Number: {dayNumber}");
```

```
////switch is preferred when num of cases is greater than 5 and mapping discrete values is needed
```

```
#endregion
```

```
#region c13
```

```
//int sum1 = SumArray(10, 20, 30, 40);
```

```
//Console.WriteLine($"Sum is:{sum1}");
```

```
//int[] myArray = { 5, 15, 25 };
```

```
//int sum2 = SumArray(myArray);
```

```
//Console.WriteLine($"Sum is: {sum2}");
```

```
//// Must be the last parameter,Only one params allowed,ingle-dimensional arrays only
```

```
#endregion
```

```
#region p2.1

//Console.Write("Enter a positive integer: ");

//if (int.TryParse(Console.ReadLine(), out int maxNumber) && maxNumber > 0)

//{

//    for (int i = 1; i <= maxNumber; i++)

//    {

//        Console.WriteLine(i);

//    }

//}

//else

//{

//    Console.WriteLine("Invalid input! Please enter a positive integer.");

//}

#endregion
```

```
#region p.2.2

//Console.Write("Enter an integer:");

//if (int.TryParse(Console.ReadLine(), out int number))

//{

//    for (int i = 1; i <= 12; i++)

//    {

//        int result = number * i;

//        Console.WriteLine($"{result} ");

//    }

//}

//else

//{

//    Console.WriteLine("Invalid input! Please enter a valid integer.");

//}

#endregion
```

```
#region p2.3

//Console.Write("Enter a positive integer: ");

//if (int.TryParse(Console.ReadLine(), out int maxNumber) && maxNumber > 0)
```

```
//{  
//  for (int i = 2; i <= maxNumber; i += 2)  
//  {  
//      Console.Write($"{i} ");  
//  }  
//}  
//else  
//{  
//  Console.WriteLine("Invalid input! Please enter a valid positive integer.");  
//}  
#endregion
```

#region p2.4

```
//Console.Write("Enter the base number: ");  
//int.TryParse(Console.ReadLine(), out int baseNum);
```

```
//Console.Write("Enter the power:");  
//int.TryParse(Console.ReadLine(), out int expNum);
```

```
//long result = 1;  
//for (int i = 1; i <= expNum; i++)
```

```
//{  
//  result *= baseNum;  
//}
```

```
//Console.WriteLine($"Output: {result}");  
#endregion
```

#region p2.5

```
//Console.Write("Enter a string");  
//string input = Console.ReadLine();  
//for (int i = input.Length - 1; i >= 0; i--)
```

```
//{  
//  Console.Write(input[i]);  
//}
```

#endregion

```

#region p2.6

//Console.Write("Enter an integer:");

//if (int.TryParse(Console.ReadLine(), out int number))

//{

//    int reversedNumber = 0;

//    int temp = number;

//    while (temp != 0)

//    {

//        int remainder = temp % 10;

//        reversedNumber = (reversedNumber * 10) + remainder;

//        temp /= 10;

//    }

//    Console.WriteLine(reversedNumber);

//}

//else

//{

//    Console.WriteLine("Invalid input! Please enter a valid integer.");

//}

#endregion

```

```

#region p2.7

//Console.Write("Enter the size of the array:");

// if (int.TryParse(Console.ReadLine(), out int size) && size > 0)

// {

//    int[] numbers = new int[size];

//    Console.WriteLine($"Enter {size} integers:");

//    for (int i = 0; i < size; i++)

//    {

//        Console.Write($"Element {i} :");

//        int.TryParse(Console.ReadLine(), out numbers[i]);

//    }

//    int maxDistance = 0;

//    for (int i = 0; i < size; i++)

//    {

```

```

//    for (int j = i + 1; j < size; j++)
//    {
//        if (numbers[i] == numbers[j])
//        {
//            int distance = j - i - 1;

//            if (distance > maxDistance)
//            {
//                maxDistance = distance;
//            }
//        }
//    }

// Console.WriteLine($"Longest distance between two matching elements:{maxDistance} cells");
// }

//else
//{
// Console.WriteLine("Invalid size!");
//}

#endregion

#region p2.8
//Console.Write("Enter a sentence:");
//string input = Console.ReadLine();
//string[] words = input.Split(' ');
//Array.Reverse(words);
//string reversedSentence = string.Join(" ", words);
//Console.WriteLine($"Output: {reversedSentence}");
#endregion
}
}
}

```

وأنا بتعلم #C اتعرفت على مفهومين مهمين في التعامل بين الـ Value Types والـ Reference Types وهما Boxing و Unboxing.

Boxing

هو تحويل الـ Value Type إلى Reference Type.

مثال بسيط:

```
;int number = 10
```

```
;object obj = number
```

هنا قيمة number اتحولت إلى Object.

Unboxing

هو العكس، بنرجع الـ Object إلى الـ Value Type الأصلي.

```
;int number = (int)obj
```

أهم المميزات

- يسمح باستخدام الـ Value Types مع الـ Reference Types.

- يساعدنا نتعامل مع أنواع مختلفة من البيانات باستخدام object.

- كان مهم في التعامل مع Collections غير Generic.

أهم العيوب

- Boxing و Unboxing ممكن يقللوا الـ Performance بسبب عمليات التحويل ونسخ البيانات.

- Boxing بيعمل Object جديد في الـ Heap، وده ممكن يزيد الضغط على الـ Garbage Collector.

- Unboxing محتاج Casting للنوع الصحيح، ولو النوع مختلف ممكن يحصل Runtime Error.

الخلاصة

Reference Type إلى Boxing → Value Type

Value Type إلى Unboxing → Reference Type

وفهم الفرق بينهم ساعدني أفهم أكثر طريقة تعامل #C مع الـ Memory والـ Data Types.

#CSharp #DotNet #Boxing #Unboxing #Programming #DEPI #SoftwareDevelopment #LearningJourney#

بنستخدم الـ Custom Exception لما نكون عايزين نهندل Business Rule Violation مش مجرد Technical/System Error ، عشان الكود يعبر عن مجال السيستم ونقدر نعمله catch منفصل ونعرض رسالة دقيقة.

(Business Case):

في نظام البنوك أو المحافظ الإلكترونية، لو مستخدم بيحاول يسحب 5000 جنيه ورصيده الحالي 2000 جنيه بس.

هنا السيستم مقتولش بـ Technical Exception زي NullReferenceException أو DivideByZeroException لأن الحساب موجود والأرقام صحيحة، لكن دي قاعدة بيزنس بتقول: "ممنوع السحب بمتطلب أكبر من الرصيد

فينعمل احنا InsufficientFundsException

الـ ApplicationException هو كلاس مبني جوه .NET كان الهدف الأساسي منه إن كل الأخطاء النابعة من التطبيق نفسه (App-specific / Custom Errors) تورث منه، بينما الـ SystemException مخصص للأخطاء اللي بتضرب من الـ CLR أو الـ Framework نفسه (زي NullReferenceException أو IndexOutOfRangeException).